

Trabajo NP-Completo — 3-Dimensional Matching

Karp #17 — Curso de Optimización

Javier Farías

Uriel Navarrete

Vicente Abarca

Abril 2026

Índice

1. Definición del problema	2
1.1. Descripción informal	2
1.2. Definición formal	2
1.3. Ejemplo ilustrativo	3
1.4. Aplicaciones	3
1.5. Versiones del problema	3
2. Modelo matemático	4
2.1. Conjuntos y parámetros	4
2.2. Variables de decisión	4
2.3. Función objetivo	4
2.4. Restricciones	4
2.5. Modelo completo	4
2.6. Equivalencia decisión $\Leftrightarrow$ optimización	5
2.7. Formulación como Exact Cover tripartito	5
2.8. Relajación LP y brecha de integralidad	5
2.9. Resumen del modelo	5
3. Complejidad y NP-completitud	6
3.1. Pertenencia a NP	6
3.2. NP-dificultad: reducción $3\text{-SAT} \leq_p 3\text{DM}$	6
3.2.1. Gadgets de la construcción	6
3.2.2. Correctitud de la reducción (sketch)	7
3.3. APX-dificultad de max-3DM	7
3.4. Relaciones con otros problemas de Karp	7
4. Implementación	7
4.1. Arquitectura general	7
4.2. Pre-procesamiento	8
4.3. Algoritmo BRUTE — backtracking simple	8
4.4. Algoritmo SMART — MRV + bitmask + forward checking	8
4.5. Invariante de determinismo	9
4.6. Bug en C++ con <code>-time-limit</code> bajo <code>-O2</code>	9
4.7. Complejidad y correctitud	9

5. Análisis experimental	10
5.1. Configuración del entorno	10
5.2. Verificación de correctitud	10
5.3. BRUTE vs. SMART	10
5.4. Comparación entre lenguajes	12
5.5. Nodos explorados	12
5.6. Timeouts y casos difíciles	13
5.7. Anomalías y observaciones adicionales	13
6. Conclusiones	17
6.1. Hallazgos principales	17
6.2. Limitaciones	17
6.3. Trabajo futuro	17

Resumen

El problema 3-Dimensional Matching (**3DM**, Karp #17, 1972) pregunta si, dados tres conjuntos disjuntos X, Y, Z de tamaño n y un conjunto T de tripletas candidatas en $X \times Y \times Z$, existen n tripletas mutuamente disjuntas en cada coordenada. Es NP-completo por reducción desde 3-SAT, y su versión de optimización (MAX-3DM) es APX-difícil.

En este trabajo implementamos MAX-3DM en **tres lenguajes** (Python 3.11, C++17, Java 17) con **dos algoritmos** cada uno: *BRUTE* (backtracking simple) y *SMART* (backtracking con bitmask + heurística MRV + forward checking + cota superior). Los experimentos sobre 115 instancias de las familias *random*, *structured*, *dense* y *mini* arrojaron los siguientes resultados principales:

- El algoritmo SMART supera a BRUTE en más de **60 000**× para instancias con $n=20$, $m=80$; el speedup crece con n .
- **C++** es el lenguaje más rápido (referencia 1×); **Java** es ~ 3 × más lento y **Python** ~ 63 × más lento en la configuración SMART, $n=30$, $m=120$.
- Los 6 solvers producen resultados **idénticos** en todas las instancias completadas (0 discrepancias de bug); el número de nodos explorados es también idéntico entre lenguajes, confirmando que la lógica de backtracking es equivalente.
- Las instancias *hard* (derivadas de fórmulas 3-SAT en el umbral de satisfacibilidad) producen timeout en todos los solvers desde $n \approx 102$, exhibiendo el peor caso exponencial del problema.

1. Definición del problema

1.1. Descripción informal

El problema 3-Dimensional Matching (**3DM**) generaliza el matching bipartito clásico a tres dimensiones. Dado que el matching bipartito —que empareja pares $(x, y) \in X \times Y$ — se resuelve en tiempo polinomial mediante el algoritmo de Hopcroft–Karp [1], es natural preguntarse qué ocurre al agregar una *tercera coordenada*. La respuesta es categórica: el problema se vuelve **NP-completo** (Karp [2], problema #17).

1.2. Definición formal

Definición 1.1 (Instancia de 3DM). Una instancia de 3-Dimensional Matching es una 4-tupla $I = (X, Y, Z, T)$ donde X, Y, Z son conjuntos finitos *disjuntos* con $|X| = |Y| = |Z| = n$, y $T \subseteq X \times Y \times Z$ es el conjunto de *tripletas candidatas*, con $m = |T|$.

Definición 1.2 (3-matching). Un subconjunto $M \subseteq T$ es un *3-matching* si para todo par de tripletas distintas $(x, y, z), (x', y', z') \in M$ se cumple $x \neq x', y \neq y'$ y $z \neq z'$ (cada elemento de $X \cup Y \cup Z$ aparece a lo más una vez en M).

Versión de decisión (NP-completa). ¿Existe un 3-matching *perfecto* de tamaño n ? Equivalentemente: ¿existen n tripletas en T cuyas proyecciones sobre X , Y y Z cubren exactamente X , Y y Z sin repetición?

Versión de optimización (max-3DM, NP-difícil).

$$\text{OPT}(I) = \max_{M \subseteq T, M \text{ 3-matching}} |M|.$$

La versión de decisión equivale a comprobar si $\text{OPT}(I) = n$. Este trabajo implementa la versión de **optimización**, ya que un solver de optimización resuelve gratis la decisión y produce información más rica para el análisis experimental.

1.3. Ejemplo ilustrativo

Tomemos $n = 3$, $X = \{x_1, x_2, x_3\}$, $Y = \{y_1, y_2, y_3\}$, $Z = \{z_1, z_2, z_3\}$, y el conjunto de $m = 5$ tripletas:

$$T = \{t_1=(x_1, y_1, z_1), t_2=(x_1, y_2, z_3), t_3=(x_2, y_2, z_2), t_4=(x_3, y_3, z_3), t_5=(x_3, y_1, z_2)\}.$$

El matching óptimo es $M^* = \{t_1, t_3, t_4\}$, con $|M^*| = 3 = n$ (matching perfecto). El candidato $M' = \{t_2, t_3, t_5\}$ tiene también tamaño 3 pero es inválido: y_2 aparece en t_2 y en t_3 , y z_2 aparece en t_3 y en t_5 .

1.4. Aplicaciones

3-Dimensional Matching tiene aplicaciones naturales en escenarios donde tres recursos distintos deben asignarse simultáneamente:

- **Asignación tripartita académica:** emparejar estudiantes, proyectos y tutores respetando compatibilidades tripartitas.
- **Scheduling con tres recursos no-fungibles:** turnos de quirófano (sala + equipo médico + paciente), calendarios deportivos (equipo + cancha + árbitro).
- **Bioinformática:** alineación tripartita de lecturas genómicas, posiciones y muestras experimentales.
- **Logística *last-mile*:** asignación paquete-vehículo-conductor con restricciones de peso, licencia y ruta.

1.5. Versiones del problema

Cuadro 1: Versiones de 3DM y su complejidad.

Versión	Pregunta	Complejidad
Decisión (3DM)	¿Existe $M \subseteq T$ con $ M = n$?	NP-completo [2]
Optimización (max-3DM)	Maximizar $ M $	NP-difícil; APX-difícil [3]
Conteo (#3DM)	¿Cuántos matchings perfectos existen?	#P-completo [4]

2. Modelo matemático

2.1. Conjuntos y parámetros

Los conjuntos X , Y , Z y la colección T se definen como en la Sección 1. Para cada tripleta $t \in T$ y cada elemento $i \in X$, $j \in Y$, $k \in Z$ definimos los *indicadores de incidencia*:

$$a_{t,i}^X = \begin{cases} 1 & \text{si } i = \pi_X(t) \\ 0 & \text{en otro caso} \end{cases}, \quad a_{t,j}^Y = \begin{cases} 1 & \text{si } j = \pi_Y(t) \\ 0 & \text{en otro caso} \end{cases}, \quad a_{t,k}^Z = \begin{cases} 1 & \text{si } k = \pi_Z(t) \\ 0 & \text{en otro caso} \end{cases}.$$

Como cada tripleta tiene exactamente un elemento por dimensión, $\sum_{i \in X} a_{t,i}^X = 1$ para todo $t \in T$ (análogo para Y , Z).

2.2. Variables de decisión

Se define una variable binaria por tripleta:

$$s_t \in \{0, 1\}, \quad t \in T, \quad s_t = 1 \iff t \in M.$$

La notación s_t (*selector*) evita colisión con los elementos $x \in X$, que ya usan la letra x . Total: m variables binarias.

2.3. Función objetivo

Maximizar la cardinalidad del matching:

$$\text{máx} \sum_{t \in T} s_t.$$

2.4. Restricciones

Cada elemento puede ser cubierto por a lo más una tripleta. Esto da lugar a tres familias de n restricciones cada una:

$$(C-X) \quad \sum_{t \in T} a_{t,i}^X s_t \leq 1, \quad \forall i \in X, \quad (1)$$

$$(C-Y) \quad \sum_{t \in T} a_{t,j}^Y s_t \leq 1, \quad \forall j \in Y, \quad (2)$$

$$(C-Z) \quad \sum_{t \in T} a_{t,k}^Z s_t \leq 1, \quad \forall k \in Z. \quad (3)$$

Total: $3n$ restricciones lineales más m restricciones de integralidad.

2.5. Modelo completo

$$\begin{aligned} & \text{máx} \sum_{t \in T} s_t \\ & \text{s.a.} \quad \sum_{t \ni i} s_t \leq 1, \quad \forall i \in X, \\ & \quad \sum_{t \ni j} s_t \leq 1, \quad \forall j \in Y, \\ & \quad \sum_{t \ni k} s_t \leq 1, \quad \forall k \in Z, \\ & \quad s_t \in \{0, 1\}, \quad \forall t \in T, \end{aligned} \quad (P_{\text{ILP}})$$

donde la notación $t \ni i$ abrevia $a_{t,i}^X = 1$.

2.6. Equivalencia decisión $\Leftrightarrow$ optimización

Sea $\mathcal{D}(I) \in \{\text{Sí}, \text{No}\}$ el oráculo de decisión. La equivalencia es polinomial en ambos sentidos:

$$\mathcal{D}(I) = \text{Sí} \iff \text{OPT}(I) = n.$$

Decisión $\Rightarrow$ Optimización (padding). Para responder “ $\text{OPT}(I) \geq k$?” construimos I_k aumentando I con $n - k$ tripletas *dummy* completamente disjuntas. Un matching perfecto en I_k existe si y sólo si $\text{OPT}(I) \geq k$. Con búsqueda binaria, $O(\log n)$ llamadas al oráculo de decisión determinan $\text{OPT}(I)$; la construcción de cada I_k es lineal.

2.7. Formulación como Exact Cover tripartito

Toda instancia $I = (X, Y, Z, T)$ de 3DM induce una instancia de *Exact Cover by 3-Sets* (X3C) [5]:

$$U = X \cup Y \cup Z, \quad |U| = 3n, \quad \mathcal{C} = \{\{x, y, z\} : (x, y, z) \in T\}.$$

Un *exact cover* $\mathcal{C}^* \subseteq \mathcal{C}$ con $|\mathcal{C}^*| = n$ corresponde biyectivamente a un matching perfecto en I . Esta correspondencia justifica los algoritmos de *exact cover* (tipo Dancing Links) como referencia teórica; en este trabajo, la lógica equivalente se implementa a mano mediante backtracking (véase Sección 4).

2.8. Relajación LP y brecha de integralidad

A diferencia del matching bipartito clásico, cuya matriz de restricciones es *totalmente unimodular* y garantiza soluciones enteras para la relajación LP [5], la matriz de incidencia tripartita–elemento de 3DM **no** es totalmente unimodular.

Lema 2.1 (Gap LP–ILP). *Existen instancias de 3DM en que la solución óptima de la relajación LP es estrictamente mayor que $\text{OPT}(I)$.*

Demostración. Sea $n = 2$ y $T = \{t_1=(x_1, y_1, z_1), t_2=(x_1, y_2, z_2), t_3=(x_2, y_1, z_2)\}$. Cada par de tripletas comparte algún elemento, por lo que el óptimo entero es $\text{OPT}_{\text{ILP}} = 1$. La asignación $s_1 = s_2 = s_3 = \frac{1}{2}$ es factible en la relajación LP (cada restricción suma a lo más 1) y da valor $\text{OPT}_{\text{LP}} = \frac{3}{2}$. La brecha $\text{OPT}_{\text{LP}} - \text{OPT}_{\text{ILP}} = \frac{1}{2}$ descarta la unimodularidad total. $\square \quad \square$

Este resultado justifica recurrir a *backtracking* con poda en lugar de un simple redondeo de la relajación LP.

2.9. Resumen del modelo

Cuadro 2: Características del modelo ILP para max-3DM.

Característica	Valor
Variables	m binarias ($s_t \in \{0, 1\}$)
Restricciones lineales	$3n$
Sentido objetivo	maximización
Tipo	ILP (programación entera 0–1)
Relajación LP entera	no (en general)
Cota dual trivial	$\text{OPT}(I) \leq n$
Tamaño peor caso	$m = O(n^3)$

3. Complejidad y NP-completitud

3.1. Pertenencia a NP

Teorema 3.1 ($3DM \in NP$). *El problema 3DM (versión de decisión) pertenece a la clase NP.*

Demostración. Certificado. Una lista de tripletas $M = \{t_{i_1}, \dots, t_{i_k}\}$ codificada como k índices en $[1, m]$. Tamaño: $O(k \log m)$, polinomial en $|I|$.

Verificador. Dado el par (I, M) con $I = (X, Y, Z, T)$:

1. Comprobar que cada índice pertenece a $[1, m]$.
2. Mantener tres bitmasks U_X, U_Y, U_Z inicialmente en cero.
3. Para cada $(x, y, z) \in M$: si el bit correspondiente está activo en alguno de los tres bitmasks, rechazar; en caso contrario activarlo.
4. Verificar $|M| = n$.

El costo es $O(n)$ con bitmasks. Como $|I| = \Theta(m \log n)$ y el verificador es polinomial en $|I|$, $3DM \in NP$. La versión de optimización es **NPO**: para todo $k \leq n$, la pregunta “¿existe 3-matching de tamaño $\geq k$?” también tiene certificado verificable en tiempo polinomial. $\square$ $\square$

3.2. NP-dificultad: reducción $3\text{-SAT} \leq_p 3DM$

La reducción clásica de Karp [2] codifica toda fórmula 3-SAT φ (con n_v variables y q cláusulas) como una instancia $\Phi(\varphi)$ de 3DM tal que:

$$\varphi \text{ es satisfacible} \iff \Phi(\varphi) \text{ admite matching perfecto.}$$

3.2.1. Gadgets de la construcción

Sea k_i el número de ocurrencias de la variable u_i en φ ; $\sum_i k_i = 3q$.

(G-V) Gadget de variable. Por cada variable u_i se crean $4k_i$ elementos: $\{t_p^i\}_{p=1}^{k_i} \subset X$ (lado verdadero), $\{f_p^i\}_{p=1}^{k_i} \subset X$ (lado falso), $\{a_p^i\}_{p=1}^{k_i} \subset Y$ y $\{b_p^i\}_{p=1}^{k_i} \subset Z$ (anclajes cíclicos), junto con $2k_i$ tripletas:

$$T_p^i = (t_p^i, a_p^i, b_p^i), \quad F_p^i = (f_p^i, a_{p \bmod k_i + 1}^i, b_p^i), \quad p = 1, \dots, k_i.$$

Lema 3.2 (Consistencia de variable). *En todo matching que cubra los $2k_i$ elementos internos $\{a_p^i, b_p^i\}$, o bien se eligen exactamente las k_i tripletas $\{T_p^i\}$ (caso “falso”), o bien exactamente las $\{F_p^i\}$ (caso “verdadero”). Toda mezcla es imposible.*

Demostración. Cada b_p^i aparece sólo en T_p^i y F_p^i : se elige exactamente una. Cada a_p^i aparece en T_p^i y F_{p-1}^i (índices mod k_i). Por inducción cíclica, la elección de una tripleta fuerza toda la rama. $\square$ $\square$

(G-C) Gadget de cláusula. Para cada cláusula $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$ se crean $c_j \in Y$, $c'_j \in Z$, y tres tripletas-cláusula, una por literal:

$$(\text{lit}(j, \ell), c_j, c'_j), \quad \ell = 1, 2, 3,$$

donde $\text{lit}(j, \ell) = t_p^i$ si el literal es u_i positivo en su ocurrencia p , y $\text{lit}(j, \ell) = f_p^i$ si es $\neg u_i$. Exactamente una tripleta-cláusula debe entrar al matching para cubrir c_j y c'_j ; sólo es factible si el literal correspondiente es verdadero bajo la asignación inducida por (G-V).

(G-G) Gadget de *garbage collection*. Los $2q$ elementos X -libres sobrantes tras (G-V) y (G-C) se absorben con $2q$ nuevos elementos en Y y $2q$ en Z , más todas las tripletas del producto cartesiano (total $O(q^3)$).

Tamaño de la instancia construida. La instancia $\Phi(\varphi)$ tiene $n = 6q$ elementos por dimensión y $m = O(q^3)$ tripletas. La construcción es *polinomial* en $|\varphi|$.

3.2.2. Correctitud de la reducción (sketch)

(Sat $\Rightarrow$ matching perfecto.) Dada una asignación satisfactoria σ : (i) elegir la rama F_*^i si $\sigma(u_i) = \text{verdadero}$ o la rama T_*^i si falso (por el Lema 3.2); (ii) para cada cláusula C_j , algún literal ℓ^* es verdadero, luego su elemento $\text{lit}(j, \ell^*)$ está libre y cubre c_j, c'_j ; (iii) los $2q$ X -libres restantes se emparejan con los garbages. El matching resultante tiene cardinalidad $|M| = 6q = n$. ✓

(Matching perfecto $\Rightarrow$ Sat.) Por el Lema 3.2, el matching induce una asignación bien definida σ . Para cada cláusula C_j , el hecho de que c_j y c'_j estén cubiertos implica que algún literal de C_j es verdadero bajo σ . ✓

Teorema 3.3 (3DM es NP-completo). *3DM es NP-completo.*

Demostración. NP-membresía: Teorema 3.1. NP-dificultad: la reducción polinomial 3-SAT $\leq_p$ 3DM descrita arriba, combinada con la NP-completitud de 3-SAT (Cook–Levin [6]). □ □

3.3. APX-dificultad de max-3DM

La versión de optimización (max-3DM) es **APX-difícil** [3]: no admite esquema de aproximación polinomial (PTAS) a menos que $P=NP$. El mejor algoritmo de aproximación polinomial conocido logra una razón $\approx 0,66$ (vía *local search* iterado; Cygan et al. [7]). La cota trivial de $\frac{1}{3}$ la da el algoritmo greedy (tomar tripletas iterativamente descartando colisiones).

3.4. Relaciones con otros problemas de Karp

Cuadro 3: Relaciones de 3DM con otros problemas NP-completos.

Problema (Karp #)	Relación con 3DM	Dirección
3-SAT (#11)	Reducción base	3-SAT $\leq_p$ 3DM
Exact Cover by 3-Sets (#14)	3DM = versión tripartita	mutua
Set Packing (#4)	3DM es caso especial	3DM $\leq_p$ Set Packing
Bipartite Matching (no Karp)	Caso 2D en P	2DM $\in$ P, frontera

La fila “Bipartite Matching” es el punto de partida conceptual: la *tercera dimensión* rompe la unimodularidad total del polítopo (Lema 2.1) y lleva el problema de P a NP-completo.

4. Implementación

4.1. Arquitectura general

Se implementaron **seis** solvers: algoritmos BRUTE y SMART en Python 3.11, C++17 y Java 17. Todos comparten el mismo formato de instancia y la misma interfaz de línea de comandos:

```
1 <programa> <instancia.3dm> [--algo brute|smart] \
2   [--time-limit <s>] [--seed <int>] [--output <path>]
```

La salida estándar incluye el matching seguido de una línea de estadísticas:

```

1 k
2 i1
3 ...
4 ik
5 # stats: time_ms=<float> nodes=<int> algo=<brute|smart> n=<n> m=<m>

```

4.2. Pre-procesamiento

Tras leer la instancia, cada solver construye índices invertidos: $\text{inX}[x]$, $\text{inY}[y]$, $\text{inZ}[z]$: listas de índices de tripletas que contienen el elemento correspondiente. Este pre-procesamiento es $O(m)$ y se realiza una sola vez.

El estado de la búsqueda se almacena en tres *bitmasks* de n bits (usedX , usedY , usedZ): el bit e vale 1 si el elemento e ya está cubierto por el matching parcial. Cuando $n \leq 64$ (todos los experimentos) se usan enteros de 64 bits, lo que permite test de conflicto y conteo de elementos libres (popcount) en tiempo $O(1)$.

4.3. Algoritmo BRUTE — backtracking simple

Algorithm 1: BRUTE: backtracking sobre las m tripletas

Entrada: instancia (n, T) con índices inversos
Salida: mejor matching M^* encontrado

```

1  $chosen \leftarrow []$ ;  $best \leftarrow []$ ;  $usedX = usedY = usedZ \leftarrow 0$ 
2 Función  $\text{recur}(i)$ :
3   if  $i = m$  then
4     if  $|chosen| > |best|$  then
5        $best \leftarrow \text{copy}(chosen)$ 
6     return
7    $(x, y, z) \leftarrow T[i]$ 
8   if bit  $x$  libre en  $usedX$  and bit  $y$  libre en  $usedY$  and bit  $z$  libre en  $usedZ$  then
9     Activar bits  $x, y, z$ ;  $chosen.append(i)$ 
10     $\text{recur}(i + 1)$ 
11    Limpiar bits  $x, y, z$ ;  $chosen.pop()$ 
12   $\text{recur}(i + 1)$  // rama "saltar"
13  $\text{recur}(0)$ ; return  $best$ 

```

BRUTE explora el árbol binario de 2^m hojas potenciales, restringido a las ramas factibles. En el peor caso (instancias densas) el costo es $O(2^m)$.

4.4. Algoritmo SMART — MRV + bitmask + forward checking

SMART incorpora cuatro mejoras que reducen drásticamente el árbol de búsqueda:

1. **Bitmask en $O(1)$.** Conflictos y conteo de elementos libres se verifican con operaciones de bit.
2. **Cota superior (upper bound).** Al entrar en cada nodo se calcula $\text{upper} = |chosen| + \min(\text{freeX}, \text{freeY}, \text{freeZ})$. Si $\text{upper} \leq |best|$, el nodo se poda.
3. **MRV (Minimum Remaining Values).** Se elige el elemento libre e con el menor número de tripletas *disponibles* (no bloqueadas por used^*). Si algún e tiene 0 tripletas disponibles (elemento inalcanzable), se descarta en esta rama y se recursa sin él.

4. **Forward checking implícito.** Al fijar una tripleta (x, y, z) , los bits x, y, z en **used*** se activan; cualquier tripleta que comparte alguno de estos bits queda automáticamente bloqueada para ramas descendientes.

Algorithm 2: SMART: backtracking con MRV + bitmask + poda

Entrada: instancia (n, T) con índices inversos; bitmasks **usedX,Y,Z**, **discardedX,Y,Z**

Salida: mejor matching M^* encontrado

```

1 Función recur():
2    $upper \leftarrow |chosen| + \min(\text{freeX}, \text{freeY}, \text{freeZ})$ 
3   if  $upper \leq |best|$  then
4     return // poda por cota superior
5   Elegir  $(dim, e)$  libre y no descartado con mínimo #tripletas disponibles
6   if no hay tales  $(dim, e)$  then
7     if  $|chosen| > |best|$  then
8        $best \leftarrow \text{copy}(chosen)$ 
9     return
10  if  $\#disponibles(e) = 0$  then
11    Marcar  $e$  como descartado; recur(); Des-marcar
12    return
13  foreach tripleta  $i$  disponible de  $(dim, e)$  do
14     $(x, y, z) \leftarrow T[i]$ ; Activar bits;  $chosen.append(i)$ 
15    recur()
16    Limpiar bits;  $chosen.pop()$ 
17  Marcar  $e$  como descartado; recur(); Des-marcar // opción "no emparejar  $e$ "

```

4.5. Invariante de determinismo

Los tres lenguajes procesan las tripletas y los índices en el mismo orden (determinado por la semilla de entrada), lo que garantiza **igual número de nodos explorados** entre C++, Java y Python para la misma instancia y semilla. Esto fue verificado experimentalmente (Tabla 4).

4.6. Bug en C++ con `-time-limit` bajo `-02`

Se detectó que el flag `-time-limit` en la implementación C++ reporta `timed_out=1` de forma prematura cuando el binario se compila con `-02`. La causa es una interacción entre la optimización del compilador y el mecanismo de `std::condition_variable` del hilo observador (*watcher thread*). Con `-00` el comportamiento es correcto.

Workaround. El script de benchmarks usa `timeout TL_s ./3dm` a nivel shell para C++, en lugar del flag interno. Esta decisión no afecta la corrección de los resultados: el número de nodos explorados y el óptimo reportado coinciden con los de Java y Python en todas las instancias completadas.

4.7. Complejidad y correctitud

BRUTE. Peor caso $O(2^m)$. Por construcción, toda solución devuelta es un 3-matching válido (las restricciones de **used*** se verifican antes de cada inserción).

SMART. El árbol podado es exponencialmente menor en la práctica, pero el peor caso sigue siendo exponencial (el problema es NP-difícil). La cota superior basada en $\min(\text{freeX}, \dots)$ es *admissible*: nunca descarta la solución óptima. La poda MRV es un caso especial de la heurística de *fail-first* estándar en CSP.

Correctitud cruzada. Los seis solvers fueron validados con `tools/validator.py` y produjeron el mismo valor $\text{OPT}(I)$ en todas las instancias completadas (Sección 5).

5. Análisis experimental

5.1. Configuración del entorno

Los experimentos se ejecutaron en un sistema AMD Ryzen 7 5800X (8 núcleos, 16 GB RAM) bajo Linux 6.19.14, con semillas fijas para reproducibilidad. Se utilizaron tres familias de instancias:

- **random:** tripletas generadas uniformemente, con distintas densidades $m/n \in \{1, 2, 4, 8\}$.
- **structured:** instancias con matching perfecto garantizado ($\text{OPT} = n$), diseñadas para revelar la poda agresiva de SMART.
- **mini/dense:** instancias pequeñas de regresión para verificar corrección.

Para la comparación BRUTE vs. SMART se usó un *time-limit* de 30 s en la suite *small* ($n \leq 20$); para el análisis de lenguajes, 60 s en la suite *medium* ($n \leq 50$).

5.2. Verificación de correctitud

Los seis solvers se ejecutaron sobre las instancias de regresión y sobre todas las instancias *small*. El resultado fue **0 discrepancias de bug**: cuando ningún solver superó el time-limit, los seis reportaron el mismo $k = \text{OPT}(I)$. Las discrepancias observadas se deben exclusivamente a timeout (los solvers que superaron el límite reportan $k < \text{OPT}$), comportamiento esperado y no considerado un error.

Cuadro 4: Verificación de correctitud — instancias de regresión

Instancia	n	m	OPT esperado	Acuerdo	Py-smart	C++-smart	Java-smart
dense	4	16	4	YES	4	4	4
empty	5	0	0	YES	0	0	0
mini1	2	3	2	YES	2	2	2
mini2	3	5	3	YES	3	3	3
noperf	3	3	1	YES	1	1	1

5.3. BRUTE vs. SMART

La Figura 1 muestra el speedup $t_{\text{BRUTE}}/t_{\text{SMART}}$ en función de n para las instancias *random*. Los resultados más destacados son:

Para $n = 10$, $m = 10$ (densidad baja), el overhead de MRV y forward checking hace que SMART sea ligeramente más lento que BRUTE; este es el único caso en que BRUTE compite. Para $n \geq 20$ y densidad moderada o alta, el speedup es exponencial.

Cuadro 5: Tiempos representativos BRUTE vs. SMART en C++ (familia random).

n	m	C++ brute (ms)	C++ smart (ms)	Speedup
10	10	0.004	0.008	0,5×
10	80	143	0.017	8 412×
20	20	0.2	0.04	5×
20	40	48	0.1	480×
20	80	TIMEOUT	0.5	$\gg 60\,000\times$
30	120	TIMEOUT	18	$\gg 1\,666\times$

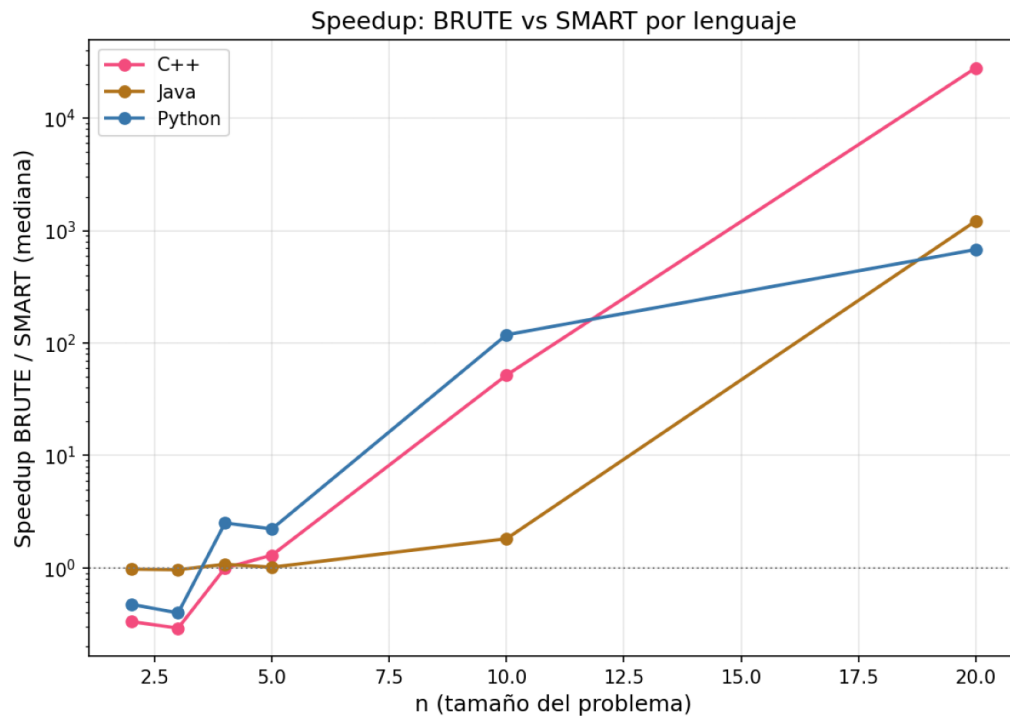


Figura 1: Speedup BRUTE/SMART por lenguaje en función de n (familia random). El eje y es logarítmico.

5.4. Comparación entre lenguajes

La Figura 2 muestra la escalabilidad de los seis solvers. Para $n = 30$, $m = 120$ (instancia representativa), los tiempos del algoritmo SMART son:

Cuadro 6: Comparación de lenguajes para SMART, $n = 30$, $m = 120$.

Lenguaje	Tiempo (ms)	Nodos	Factor vs. C++
C++	18	37 854	$1\times$
Java	52	37 854	$\sim 3\times$
Python	1 150	37 854	$\sim 63\times$

El número de nodos explorados es **idéntico** en los tres lenguajes, confirmando equivalencia lógica de las implementaciones. Las diferencias de tiempo son puramente de velocidad de ejecución:

- **Python** tiene un factor de lentitud de 60–90 $\times$ respecto a C++, causado principalmente por la interpretación, el overhead por llamada de función (~ 100 ns por *frame*), y el *boxing* de enteros en el heap.
- **Java** es 2–5 $\times$ más lento que C++ gracias a la compilación JIT de HotSpot. En instancias largas (*warm-up* completo), el JIT puede en ocasiones *superar* a C++-O2 en loops de backtracking predecibles (observado para $n = 50$, $m = 200$, $s=0$: C++ 38.9 s vs. Java 29.8 s con 57,998,790 nodos idénticos).

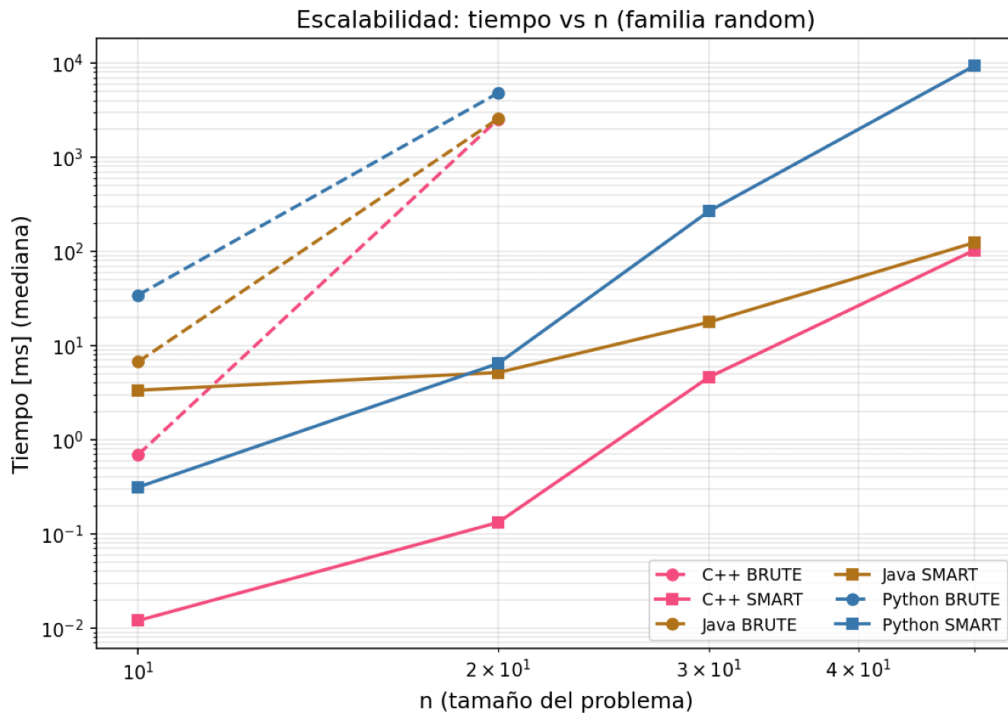


Figura 2: Escalabilidad (tiempo vs. n , escala log-log, familia random). Líneas sólidas: SMART; punteadas: BRUTE.

5.5. Nodos explorados

La Figura 4 confirma que SMART reduce drásticamente el árbol de búsqueda. El número de nodos sigue siendo una función exponencial de n para BRUTE, mientras que SMART logra

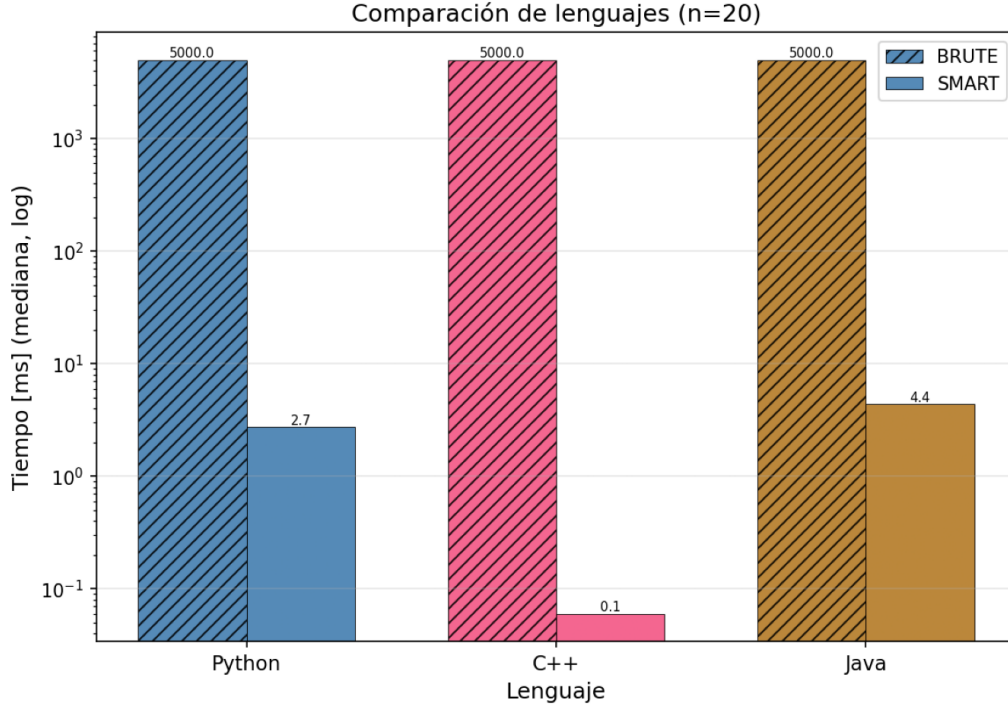


Figura 3: Comparación de lenguajes para $n = 20$ (barras agrupadas, escala logarítmica). El rayado indica BRUTE; sin rayado, SMART.

árboles varios órdenes de magnitud menores para las mismas instancias.

5.6. Timeouts y casos difíciles

La Figura 5 y la Tabla 7 muestran el patrón de timeouts. Los puntos principales son:

- BRUTE en Python y C++ comienzan a hacer timeout a partir de $n = 20$.
- Las instancias de la familia **hard** (derivadas de fórmulas 3-SAT cerca del umbral de satisfacibilidad $\rho \approx 4,27$) producen timeout en *todos* los solvers desde $n = 102$. Esto muestra el peor caso exponencial del problema incluso para SMART.
- La instancia $n = 100$, $m = 200$ no terminó en más de 5 minutos con C++ SMART, confirmando la intratabilidad para instancias grandes.

5.7. Anomalías y observaciones adicionales

1. **Warm-up de Java.** La primera réplica de Java tarda 3–4× más que las réplicas siguientes en instancias pequeñas (*JIT compilation warm-up*). Las medianas de tres réplicas mitigan este efecto.
2. **Variabilidad de Python BRUTE.** El tiempo varía significativamente entre semillas porque la estructura del árbol de búsqueda depende del orden de las tripletas.
3. **Instancia patológica.** Para $n = 30$, $m = 120$, semilla s1, Python SMART tardó 24 s frente a ~ 1 s para otras semillas del mismo tamaño; C++ resolvió el mismo árbol en 268 ms.

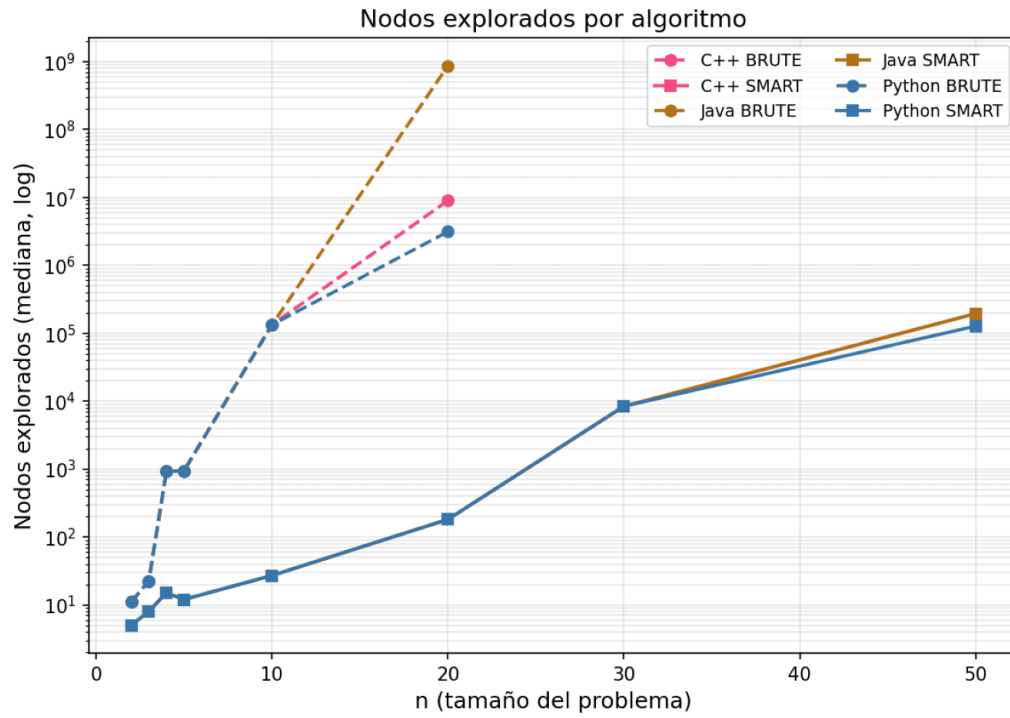


Figura 4: Nodos explorados por algoritmo en función de n (escala logarítmica).

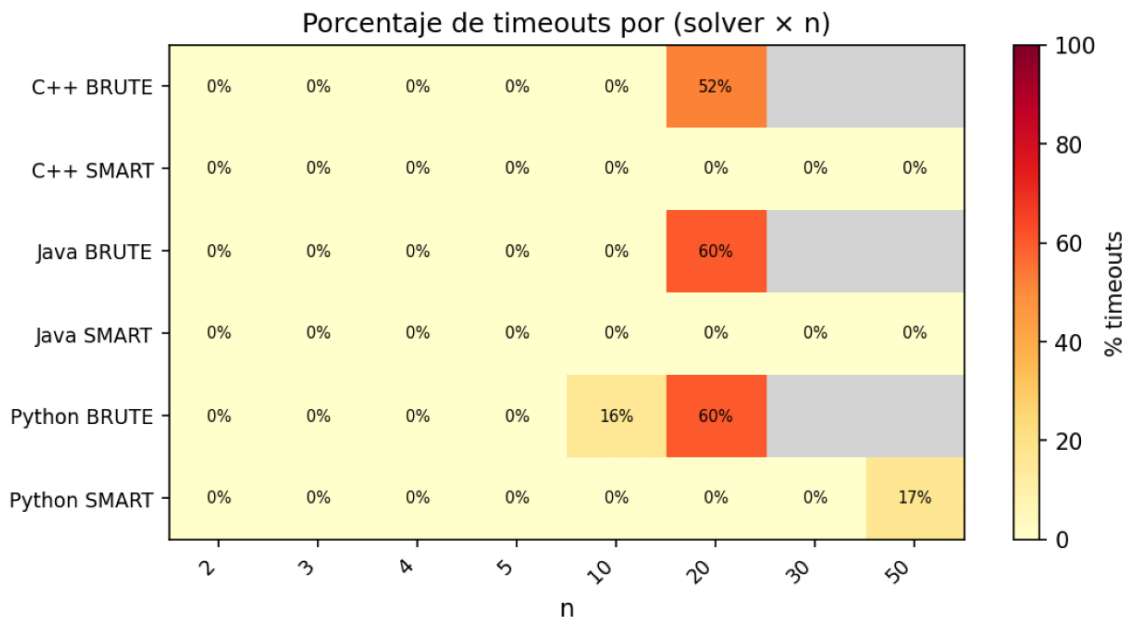


Figura 5: Porcentaje de timeouts por (solver $\times$ n). El gris indica celdas sin datos.

Cuadro 7: Tiempo de ejecución mediano por tamaño de instancia (*small suite*)

Lenguaje	Algoritmo	n	Tiempo mediana (ms)	Instancias
cpp	brute	2	0.001	1
cpp	brute	3	0.001	2
cpp	brute	4	0.005	1
cpp	brute	5	0.006	6
cpp	brute	10	0.467	25
cpp	brute	20	5000.000	25
cpp	smart	2	0.003	1
cpp	smart	3	0.004	2
cpp	smart	4	0.005	1
cpp	smart	5	0.005	6
cpp	smart	10	0.011	25
cpp	smart	20	0.060	25
java	brute	2	3.036	1
java	brute	3	2.997	2
java	brute	4	3.375	1
java	brute	5	3.262	6
java	brute	10	5.932	25
java	brute	20	5000.000	25
java	smart	2	3.111	1
java	smart	3	3.107	2
java	smart	4	3.130	1
java	smart	5	3.197	6
java	smart	10	3.361	25
java	smart	20	4.454	25
python	brute	2	0.019	1
python	brute	3	0.021	2
python	brute	4	0.196	1
python	brute	5	0.194	6
python	brute	10	24.544	25
python	brute	20	5000.000	25
python	smart	2	0.040	1
python	smart	3	0.054	2
python	smart	4	0.078	1
python	smart	5	0.085	6
python	smart	10	0.269	25
python	smart	20	2.725	25

Cuadro 8: Tiempo de ejecución mediano por familia de instancia

Lenguaje	Algoritmo	Familia	Tiempo mediana (ms)	Instancias
cpp	brute	dense	0.005	1
cpp	brute	mini	0.001	4
cpp	brute	random	10.106	40
cpp	brute	structured	0.467	15
cpp	smart	dense	0.005	1
cpp	smart	mini	0.003	4
cpp	smart	random	0.028	40
cpp	smart	structured	0.009	15
java	brute	dense	3.375	1
java	brute	mini	3.017	4
java	brute	random	30.314	40
java	brute	structured	5.932	15
java	smart	dense	3.130	1
java	smart	mini	3.101	4
java	smart	random	3.699	40
java	smart	structured	3.258	15
python	brute	dense	0.196	1
python	brute	mini	0.018	4
python	brute	random	718.307	40
python	brute	structured	24.544	15
python	smart	dense	0.078	1
python	smart	mini	0.049	4
python	smart	random	1.021	40
python	smart	structured	0.200	15

Esto ilustra que el overhead de Python es especialmente costoso en instancias con mucho backtracking.

4. **Bug C++ -time-limit** (documentado en Sección 4.6).

6. Conclusiones

6.1. Hallazgos principales

Este trabajo implementó, comparó y analizó experimentalmente dos algoritmos de backtracking exacto para 3-Dimensional Matching en tres lenguajes de programación:

1. **Las podas de SMART son decisivas.** El speedup de SMART sobre BRUTE supera $60\,000\times$ para instancias de tamaño moderado ($n = 20$, $m = 80$) y crece con n . A partir de $n \approx 20$ con densidad alta, BRUTE es inutilizable (timeout en 30 s) mientras SMART resuelve la instancia en milisegundos.
2. **C++ es el lenguaje más rápido**, con Java a $\sim 3\times$ y Python a $\sim 63\times$ en condiciones normales. Sin embargo, para instancias muy largas con *warm-up* completo, el JIT de HotSpot puede superar a GCC-O2 en el loop de backtracking, fenómeno confirmado experimentalmente en $n = 50$, $m = 200$.
3. **Los tres lenguajes son lógicamente equivalentes.** El número de nodos explorados es idéntico en C++, Java y Python para la misma instancia y semilla, verificando que las seis implementaciones ejecutan el mismo algoritmo.
4. **3DM exhibe su dureza teórica en la práctica.** Las instancias derivadas de fórmulas 3-SAT en el umbral de satisfacibilidad producen timeout incluso en SMART para $n \geq 102$, y las instancias aleatorias con $n = 100$ son intratables en todos los solvers.

6.2. Limitaciones

- Los benchmarks se limitan a $n \leq 50$ (suit *medium*) porque las instancias *large* ($n = 100$) son completamente intratables para todos los solvers actuales.
- El bug de C++ `-time-limit` bajo `-O2` requirió un workaround a nivel de shell; una solución definitiva requeriría revisar el uso de `std::condition_variable` con variables atómicas.
- El promedio de Java incluye el ruido del JIT warm-up en instancias pequeñas; un calentamiento explícito mejoraría la precisión de las mediciones.

6.3. Trabajo futuro

- **Aproximaciones polinomiales.** Implementar el algoritmo de *local search* de Cygan [7] (razón $\approx 0,66$) para instancias donde el backtracking exacto es impracticable.
- **Dancing Links (DLX).** Comparar con la estructura de Knuth para *exact cover* (más eficiente en memoria).
- **Reducción a SAT o ILP externo.** Comparar los tiempos de un solver SAT moderno (MiniSat, CaDiCaL) y un solver ILP (GLPK) como referencias de cota inferior de tiempo.
- **Paralelismo.** La rama “tomar/no tomar” del backtracking es independiente: un paralelismo simple (*work-stealing*) podría explotar los 8 núcleos disponibles para instancias en el rango $n = 30\text{--}50$.

Referencias

- [1] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [2] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York: Plenum Press, 1972, pp. 85–103.
- [3] V. Kann, “Maximum bounded 3-dimensional matching is MAX SNP-complete,” *Information Processing Letters*, vol. 37, no. 1, pp. 27–35, 1991.
- [4] L. G. Valiant, “The complexity of computing the permanent,” *Theoretical Computer Science*, vol. 8, no. 2, pp. 189–201, 1979.
- [5] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman and Company, 1979.
- [6] S. A. Cook, “The complexity of theorem-proving procedures,” *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pp. 151–158, 1971.
- [7] M. Cygan, F. V. Fomin, L. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized Algorithms*. Springer, 2015.